Your own skill

When something should be a skill, how to write one with Claude, and how to tell it works

Florian Gärtner

Where we are today

	What Day 1 taught
	What Claude needs in every session
	What Claude needs only sometimes
→	**Your own skill** ← you are here
	Lunch
	A history you can go back to
	Where does this file live?
	Commit first, then let Claude move things. Then GitHub
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Decide** whether something you keep telling Claude belongs in `CLAUDE.md`, a skill, a hook or a subagent.
2. **Write** a skill with Claude from a task you just did, and check its description and body.
3. **Test** a skill in a new session, with it and without it.

From something you repeat to a skill that works

Something you repeat
a procedure,
not a fact
so a skill

→

Claude writes it
from a task you
just did
or from
someone else's

→

You check it
the description
and the body

→

A new session
with the skill,
and without
did the result
change?

Seeing it load is not seeing it work.

Is it a skill at all?

What have you told Claude more than once?

Your own project has been running beside the course since yesterday. Think of a step you had to explain again, or a format you had to ask for again.

Where should it go, so you never type it again?

Facts in `CLAUDE.md`, procedures in a skill

What you keep telling Claude	Where it goes
A fact about the project: *income is in thousands of euros*	`CLAUDE.md`
A procedure: *how the survey export is cleaned*	a skill
A rule that must hold every time: *never change the raw data*	a hook
A job that runs on its own and reports back: *search the literature*	a subagent

Claude Code docs, skills page: create a skill “when a section of CLAUDE.md has grown into a procedure rather than a fact”.

Until Claude opens it, a skill costs almost nothing. `CLAUDE.md` is sent with every turn.

A skill or a subagent?

A **subagent** is a second Claude that Claude sends out with one task. It works in a context of its own and sends back a summary (block 1_5, “Send the reading away”).

	A skill	**A subagent**
What it gets	instructions, in your conversation	a task, in a context of its own
What comes back	Claude follows them while working with you	a summary
Good for	a procedure for the work you do together	a big search or check whose details you do not need

One you may have met: Claude sends out **Explore** to search a folder, and you get its summary.

Your kit brings two more. `literature-scout` searches for papers on a question and copies each reference from a page it opened. `verifier` checks that a finished task was really done.

Claude Code docs, subagents page.

Write it with Claude

Start from a task you just did

In the work slot, write yours like this:

1. Do the task with Claude, the usual way.
2. Notice what you had to explain.
3. Ask Claude: *"Turn what we just did into a skill."*
4. Read the draft, and have Claude cut what it knows anyway.
5. Try it in a new session.

Anthropic, Skill authoring best practices.

With the kit, Claude loads its `skill-builder` skill for step 3.

Or start from someone else's

Collections of skills are online, for example:

- Anthropic's own, `github.com/anthropics/skills`, "for demonstration and educational purposes only"
- one for the sciences, `github.com/K-Dense-AI/scientific-agent-skills`

Found one? Do not install it as it is. Ask Claude first:

> Read this skill and compare it with what I already have. Is it useful for me, which parts, and is it well made? Change nothing until I say so.

Then take all of it, a part, or nothing, as you did with the kit on Day 1.

Check the description

Until Claude opens a skill, it sees only the skill's name and description. **The description decides whether Claude opens it.** Read the one Claude wrote:

- Does it say **what** the skill does?
- Does it say **when** to use it, in the words you use when you ask for that task?
- Is the main use case **first**? A long description is cut short.

Description	**Claude opens it when you ask to …**
"Survey stuff."	probably never
"Cleans a raw survey export. Use when the user asks to clean, recode or import a survey file."	clean, recode or import a survey

Your kit's skills write the when as *"ALWAYS use this skill when …"*. In one test, Claude opened skills written that way more reliably.

Check the body

- **Only what Claude does not know.** Claude knows R, but not your column names.
- **Exact steps where a mistake is costly**, and room for judgment where the task is open.
- **One example** shows more than a paragraph of description.
- **The most important part first**, and long reference material in separate files the skill points to.
- **One job.** If the skill does two things, ask Claude to split it into two skills, each with its own description.
- **A list of traps** that grows: each time Claude gets the task wrong, ask it to add the mistake and how to avoid it. Your kit's `skill-builder` starts one in every skill.

Anthropic, Skill authoring best practices (the first four).

Does it work?

Seeing it load is not seeing it work

In the work slot, test your skill like this:

1. Open a **new** session and ask three things you would really ask.
2. Turn the skill off in `/skills`: highlight it, press `Space` until it is off, then `Esc`.
3. Ask the same three in another new session and compare. Then turn the skill back on.

Claude Code docs, skills page: “Seeing a skill trigger tells you Claude found it, not that it did what you intended.”

With the kit, Claude names the skills it loaded before each task, so you see whether yours was opened.

When the skill does not work

What you see	What to try
Claude never opens it	add the words you use when you ask for the task (*recode*, *survey export*) to its description, or type `/skill-name`
… and you have many skills	its description may have been cut: switch off skills you do not use, in `/skills`
Claude follows it once, then not	word the step for the whole task: *"Run the tests after every edit"*, not *"Run the tests"*
A rule in it gets skipped	a hook, if the rule must hold every time
It faded after a long session	call it again, since after compacting only its start is kept

Claude Code docs, skills page, Troubleshooting.

Your own skill

Pick a procedure, and a project Claude may read

Exercise · 40 min

⚠ **Every file Claude reads goes to the model provider.**

Take a procedure from your own project that you have explained to Claude more than once, such as cleaning an export, the format of your tables, or how you want a draft read.

Time for your own skill

Exercise

This slot is for one skill of your own. Work on what fits you. Some suggestions:

- **No skill yet?** Do the task with Claude, then have Claude turn it into a skill.
- **A skill already?** Test it with and without, in two new sessions.
- **Want to see a good one first?** Read one of the kit's skills, and ask Claude what makes its description work.

Then: what did your test show?

Discussion

1. A colleague shares a skill with your field's standard robustness checks. Do you use it?
2. Your skill cleans the survey export the way your group always does. Do you put it in the project, where your coauthors' Claude finds it too, or keep it in your own `~/.claude`?
3. Your test showed no difference between the session with the skill and the one without. Do you keep the skill?

What to take away

1. Facts go in `CLAUDE.md`, procedures in a skill, a rule that must always hold in a hook, and a job of its own to a subagent.
2. Write a skill from a task you just did, and cut what Claude knows anyway.
3. The description decides when Claude opens a skill: what it does, and when, in your words.
4. Test it in a new session, with it and without it. Seeing it load is not seeing it work.
5. Read any skill with Claude before you take it.